



Inteligencia Artificial

II. Clasificación: K vecinos más cercanos

Tarea 2

Detección de insuficiencia cardiaca

Con el repositorio heart.csv que está disponible en el siguiente enlace:

[heart-failure-prediction](https://www.kaggle.com/lakshmi25nair/heart-failure-prediction)

Realice lo siguiente en un Notebook de Jupyter:

1. Cargue los datos en Python usando Pandas y muestre los primeros 10 registros
2. Identifique que porcentaje de valores faltantes tiene cada columna
3. Muestre los valores estadísticos básicos de todas las columnas (contador, media, desviación estándar, valor mínimo, valor máximo y los cuartiles).
4. Con el siguiente código genere el histograma de todas las columnas:

```
viz_df = df.copy()
k = 0
plt.figure(figsize=(12, 15))
for col in viz_df.columns:
    plt.subplot(4, 3, k + 1)
    plt.hist(viz_df[col])
    plt.title(f"{col}")
    k += 1
plt.tight_layout()
```

5. Con el siguiente código genere el diagrama de bigotes de todas las columnas:

```
viz_df = df.select_dtypes(include=np.number)
k = 0
plt.figure(figsize=(12, 10))
for col in viz_df.columns:
    plt.subplot(3, 3, k + 1)
    plt.boxplot(viz_df[col])
    plt.title(f"{col}")
    k += 1
plt.tight_layout()
```



6. Con el siguiente código obtenga las categorías de las columnas categóricas:

```
categoricas=('Sex' , 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope')
for i in categoricas:
    print("categorias de ", i, ":", df[i].unique())
```

7. Como podrá notar sexo y ExerciseAngina solamente poseen 2 posibles valores convierta estas columnas a numéricas con el OrdinalEncoder.
8. Codifique las otras tres columnas categóricas con el OneHotEncoder, asegúrese de que al indicar el nombre de las columnas resultantes estén en orden alfabético, pues así las generará el codificador. Una vez codificadas elimínelas del dataframe.
9. Revise las dimensiones del dataframe.
10. Genere la matriz de correlación completa y en su forma tidy.
11. Separe las columnas independientes de la variable objetivo ('HeartDisease') en X,Y
12. Use la función train_test_split para separar el conjunto de datos en 80% entrenamiento y 20% prueba.
13. Escale los datos (solo Xtest y Xtrain)
14. Haga el análisis de componentes principales con 18 componentes y vea que porcentaje de varianza explican los primeros 12 componentes. Repita el análisis, pero ahora con usando solo 12.
15. Obtenga los componentes principales del conjunto X (train y test).
16. Genere un GridSearchCV para encontrar la mejor configuración (gridsearch.best_params_) para el KNeighborsClassifier usando los siguientes parámetros:

```
parameters = {"n_neighbors": range(2, 50), "weights": ('distance', 'uniform')}
```
17. Entrene el modelo GridSearchCV, haga las predicciones y obtenga el error cuadrático medio para los datos de prueba y los de entrenamiento.
18. Muestre la matriz de confusión, el puntaje F1 y el reporte de clasificación de los datos de prueba.



19. Usando 2 de los componentes principales grafique los datos reales y las predicciones del modelo para identificar visualmente que tan efectivo es el modelo.

20. Repita el procedimiento, pero ahora con los siguientes parámetros:

```
parameters = {"n_neighbors": range(3, 100), "weights": ('uniform', 'distance'),  
              "metric": ("euclidean", "manhattan", "minkowski")}
```